UNIT 2: CONDITIONAL EXECUTION AND ITERATION

Simple If:

```
if condition :
    statement

     OR

if condition :
    statement-1
    statement-2
    statement-3
```

If condition is true then statements will be executed

In [2]:
```
x=5
if x==5:
    print("Hello, Good Morning")
print("How are you!!!")
```

```
Hello, Good Morning
How are you!!!
```

In [3]:
```
x=6
if x==5:
    print("Hello, Good Morning")
print("How are you!!!")
```

```
How are you!!!
```

If-else:

```
if condition:
    Action-1

else:
    Action-2
```

if condition is true then Action-1 will be executed otherwise Action-2 will be executed.

In [4]:
```
x=5
if x==5:
    print("Hello, Good Morning")
else:
    print("hi")
print("How are you!!!")
```

```
Hello, Good Morning
How are you!!!
```

In [5]:
```
x=8
if x==5:
    print("Hello, Good Morning")
else:
    print("hi")
print("How are you!!!")
```

```
hi
How are you!!!
```

If-elif-else:

if condition 1:

    Action-1

elif condition 2:

    Action-2

elif condition 3:

    Action-3
    ….

else:

    Default Action

Based on condition the corresponding action will be executed

In [9]:
```
"""Python program to read marks from user and give grade according to
following marks grade"""
# 100-80 distinction
# 60-79 first class
# 35-59 second class
# 0-34 fail
a=int(input("Enter marks out of 100: "))
if(a>=80 and a<=100):
     print("Distinction")
elif(a<=79 and a>=60):
     print("First class")
elif(a<=59 and a>=35):
     print("Second class")
elif(a<=34):
     print("Fail")
else:
    print("enter proper data")
```

```
Enter marks out of 100: 123
enter proper data
```

Nested If:

You can have if statements inside if statements, this is called nested if statements.

In [10]:
```
x = 41
if x > 10:
    print("Above ten,")
    if x > 20:
        print("And also above 20!")
    else:
        print("but not above 20.")
```

```
Above ten,
And also above 20!
```

Iterative Statements:

If we want to execute a group of statements multiple times then we should go for Iterative statements.

Python supports 2 types of iterative statements. for loop while loop

for loop:

If we want to execute some action for every element present in some sequence (it may be string or collection) then we should go for for loop.

```
Syntax:

for x in sequence:

    Body

Where sequence can be string or any collection.

Body will be executed for every element present in the sequence.
```

In [14]:
```
#Example1: To print characters present in the given string.
s="LJIET"
for x in s :
    print(x)
```

```
L
J
I
E
T
```

In [15]:
```
#Example 2: To print Hello 5 times
for x in range(5) :
    print("Hello")
```

```
Hello
Hello
Hello
Hello
Hello
```

In [16]:
```
#Example 3: To display numbers from 0 to 5
for x in range(6) :
    print(x)
```

```
0
1
2
3
4
5
```

In [17]:
```
for x in range(3,6) :
    print(x)
```

```
3
4
5
```

In [18]:
```
for x in range(1,13,2) :
    print(x)
```

```
1
3
5
7
9
11
```

while loop:

If we want to execute a group of statements iteratively until some condition false, then we should go for while loop.

```
        Syntax:

    while  condition:
                body
```

In [19]:
```
#Example : To print numbers from 1 to 5 by using while loop
x = 1
while x <= 5:
    print(x)
    x = x+1
```

```
1
2
3
4
5
```

In [20]:
```
name=""
while name!="Arman":
      name=input("Enter Name:")
print("Thanks for confirmation")
```

```
Enter Name:aRMAN
Enter Name:Arman
Thanks for confirmation
```

Nested Loops:

Sometimes we can take a loop inside another loop,which are also known as nested loops.

In [21]:
```
for i in range(2):
    for j in range(2):
        print("i=", i,"j=", j)
```

```
i= 0 j= 0
i= 0 j= 1
i= 1 j= 0
i= 1 j= 1
```

Break Statement:

We can use break statement inside loops to break loop execution based on some condition.

In [22]:
```
for i in range(10):
    if i==7:
        print("processing is enough..plz break")
        break
    print(i)
```

```
0
1
2
3
4
5
6
processing is enough..plz break
```

Continue Statement:

We can use continue statement to skip current iteration and continue next iteration.

In [23]:
```
for i in range(10):
    if i%2==0:
        continue
    print(i)
```

```
1
3
5
7
9
```

Pass Statement:

pass is a keyword in Python.
In our programming syntactically if block is required which won't do anything then we can define that empty block with pass keyword.
Pass is an empty statement.
It is null statement.
It won't do anything.

In [29]:
```
if True:
```

```
  File "<ipython-input-29-2c8a33b52dfb>", line 1
    if True:
            ^
SyntaxError: unexpected EOF while parsing
```

In [26]:
```
if True: pass #valid syntax
```

In [28]:
```
for i in range(20):
    if i%9==0:
        print(i)
    else:
        pass
```

```
0
9
18
```

For loop with break ,else

In [31]:
```
for i in range(1, 4):
      print(i)
else:  # Executed because no break in for
      print("No Break")
```

```
1
2
3
No Break
```

In [30]:
```
for i in range(1, 4):
      print(i)
      break
else: # Not executed as there is a break
      print("No Break")
```

```
1
```

While loop with else

In [32]:
```
count = 0
while (count < 1):
    count = count+1
    print(count)
else:
    print("No Break")
```

```
1
No Break
```

In [33]:
```
count = 0
while (count < 1):
    count = count+1
    print(count)
    break
else:
    print("No Break")
```

```
1
```

In [ ]: